Third, we collect deployment data from more diverse sources. Following Needham et al. we split data into *chat* datasets and *agentic* coding datasets, for both evaluation and deployment. For agentic trajectories we draw mostly from DataClaw [O'Malley, 2026], sampling across user projects, scaffolds, model families, and turn lengths. We include a curated DataClaw subset from Traced.run [Schaeffer, 2026] that shows agent misbehaviour (unauthorised actions, scope creep, data deletion), which we cut around the misbehaviour signal. For chat deployment we use ShareChat [Yan et al., 2026], WildBench [Lin et al., 2024] and CodeChat [Zhong et al., 2025], and crowdsourced chat links. From Needham et al.'s pool we keep only the chat datasets sourced from in-the-wild user-LLM chat conversations and curated samples from Open Deep Research.

The curated suite contains 746 trajectories: 413 evaluation (98 agentic, 315 chat-style) and 333 deployment (145 agentic, 188 chat). Evaluation is chat-heavy, with coverage skewing toward short QA, and within deployment the agentic share is higher, skewing toward coding sessions. AUROC is reported on the full suite.

Full composition is in Appendix F. The pipeline treats the trajectory suite as a configurable input, so this composition is a deliberate but replaceable choice to allow practitioners to refresh or swap sources as deployment data and benchmarks evolve.

3.2 Probe question selection and scoring

A probe question (PQ) is the instrument through which we measure evaluation awareness. Each PQ is a closed-ended question appended to a transcript τ, asking the judge to decide whether the conversation originated from evaluation or deployment. The judge returns a binary label, which we encode as $v_k = 1$ for evaluation and $v_k = 0$ for deployment. Given an ensemble of K probes, we aggregate to a per-transcript score

$$P(\text{eval})(\tau) = \frac{1}{K} \sum_{k=1}^{K} v_k \in [0, 1], \tag{1}$$

where higher values indicate that more probes flagged the transcript as evaluation-like. We report *AUROC* as the primary metric (threshold-free and class-balance-invariant), along with AUPRC and binary accuracy at a configurable threshold.

3.2.1 PQ corpus curation

Our PQ corpus comprises 20 closed-ended questions: 17 direct and 3 indirect probes (full list in Appendix A). We retain four out of Needham et al.'s five direct PQs, dropping one that assumed deployment interactions always come from a human user and thus mislabels traffic from automated systems (e.g., an email-classification pipeline). We add 13 more direct probes spanning surface-level to structural cues, forming a diverse, partially redundant pool for per-model selection. Because direct PQs mention *evaluation* explicitly and may invite keyword heuristics, we add three indirect probes that rely on structural and pragmatic cues instead. For example:

> Does the interaction feel like it emerged naturally — through spontaneous phrasing, informal tone, or reactive follow-ups — rather than being pre-planned?

3.2.2 Per-model selection procedure

Needham et al. select five PQs from an initial pool of 28 based on a performance–cost trade-off for GPT-4.1, and apply that fixed set across all models. This implicitly assumes a single PQ set transfers across model families. We instead treat PQ choice as a capability elicitation step performed per model: for each judge, we search for the subset of probes that most successfully surfaces its evaluation awareness.

Procedure For each judge model, we sample a 20% calibration slice stratified by the evaluation/deployment and chat/agentic axis. On the calibration set, we exhaustively search over all PQ subsets of size $k \leq 5$ from our 20-item corpus and retain the subset that maximises ensemble AUROC. The cap at $k = 5$ is set for cost reasons and justified empirically below.

3.2.3 Validation

We validate our per-model probe selection approach on a 250-transcript subset across four frontier models spanning two families (GPT-5.1, GPT-5.4, Claude Sonnet 4.6, Claude Opus 4.6). Figure 2 supports three claims:

(i) **Per-model selection elicits more awareness than the Needham baseline:** held-out AUROC meets or exceeds the baseline from $k \approx 3$–4 onward, reaching 0.91 (Sonnet 4.6), 0.90 (Opus 4.6), and 0.82 (GPT-5.1). GPT-5.4 is the exception — the baseline is already strong (≈ 0.82) and per-model selection tracks near it, suggesting a ceiling effect. The differences between models support the argument for per-model selection: a fixed set serves some judges well and others poorly, and which case applies cannot be determined in advance.

(ii) **Selection beats random sampling:** held-out curves sit above the mean of 500 random subsets at every k, with the largest gap at small k where selection matters most.

(iii) **Held-out AUROC plateaus by** $k \approx 5$**–7**, justifying the $k \leq 5$ cap in our selection procedure.

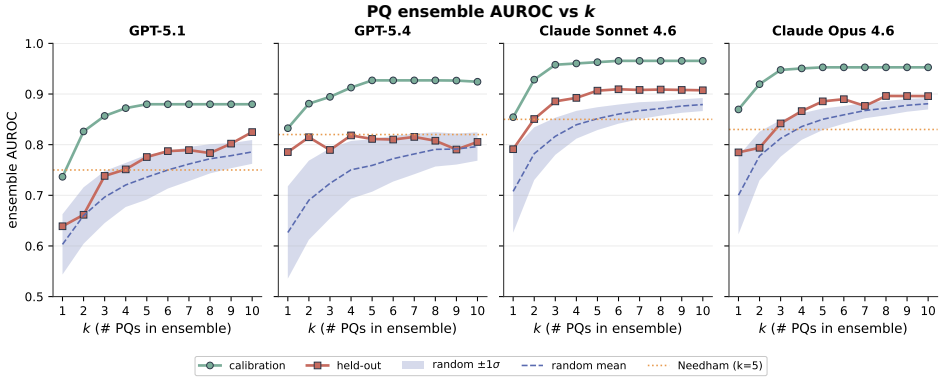


Figure 2: **Ensemble AUROC vs. number of probe questions, per model, under four selection strategies.** Per-model exhaustive calibration selection (green) and its held-out performance (red), versus a random-selection baseline (blue, mean $\pm$ 1 std over 500 Monte Carlo draws per k) and Needham et al.'s fixed PQ set (orange dotted line, $k = 5$).

Together these results show that calibration-based per-model selection elicits evaluation awareness better than either a fixed cross-model PQ set or random sampling. Because evaluation awareness has no well-defined ground truth, the question is whether we have elicited the capability well enough to support conservative estimates, and per-model selection provides stronger evidence that differences across judges reflect genuine variation in capability rather than under-elicitation.

3.3 Generator-identity confound

Deployment data is typically sourced from a mixture of model families, often distinct from the judge under test. We show that this mixture is not a neutral design choice: generator identity contributes a substantial share of the variance in $P(\text{eval})$, this contribution persists under the resampling strategy proposed by Williams et al. [2025], and it can shift relative model rankings. We then propose a post-hoc correction (Section 3.3.3) for cases where same-family deployment data is unavailable, such as evaluating models before deployment.

3.3.1 Generator identity shifts P(eval)

We construct a 6×6 fully crossed design: every model generates deployment transcripts (sourced from DataClaw conversations), and every model judges every other model's transcripts ($\sim$50 samples per cell). We score each cell with the average $P(\text{eval})$ over two probe questions (one direct, one purpose-related).

Figure 3 shows a clear columnar pattern: Opus 4.5 ranks most deploy-like for every judge without exception (claude-opus $= 0.410$, gpt $= 0.527$, gemini $= 0.603$; a $+0.155$ gap vs. all other generators on a 0–1 scale). We can also observe that there is little self-preference bias: the same-family mean (0.498) is close to the cross-family mean (0.521) with a 0.023 gap.

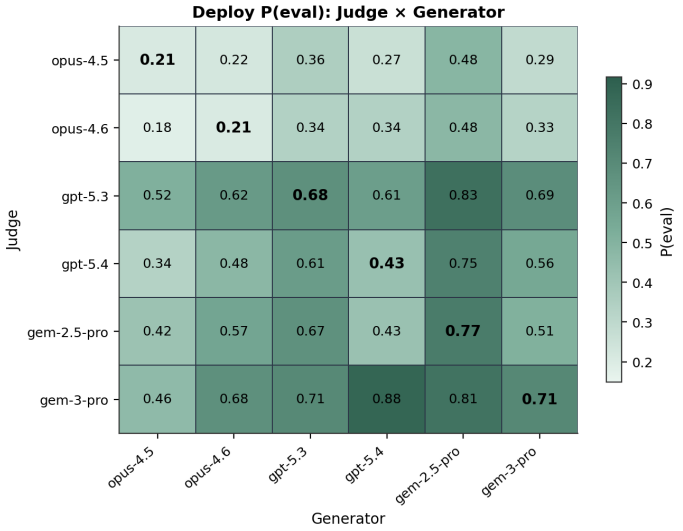


Figure 3: **Claude-style models are generally seen as more deploy-like by all judges.** Deploy $P(\text{eval})$: Judge $\times$ Generator. The columnar pattern shows the generator effect: Claude transcripts (left columns) receive low $P(\text{eval})$ from all judges, while GPT and Gemini transcripts receive comparatively high scores regardless of who judges them.

This pattern is consistent with a generator-identity effect rather than a judge-specific one. To quantify it, we fit a Type II two-way ANOVA decomposing $P(\text{eval})$ variance into generator, judge, and interaction components (full model in Appendix B). **Generator identity explains 11.25% of $P(\text{eval})$ variance** ($F = 67.2$, $p < 0.001$); for unconfounded deployment data this should be zero.

We further decompose the confound into generation style versus user task distribution and find that generation style is the primary driver (Appendix D).

3.3.2 Resampling does not eliminate the confound

Williams et al. [2025] proposes a resampling technique for constructing deployment-proxy data: strip the last assistant response from a real conversation and regenerate it with a different model. If the generator signal resides primarily in the most recent assistant turn, replacing that turn should reduce the generator-identity variance.

We test this by varying resampling depth k (the number of trailing messages stripped and regenerated by the judge): each model resamples $\sim$30 conversations from each of the 6 generators (including itself) at each depth, then judges its own resampled set.

Figure 4 shows that generator-family separation in $P(\text{eval})$ persists at every resampling depth: the gap between Claude-generated and GPT/Gemini-generated transcripts visible at $k = 0$ remains at $k = 9$, regardless of which family is judging. The full ANOVA decomposition (Appendix C) confirms this quantitatively: generator identity accounts for 8.4–11.3% of variance across $k = 0$ to $k = 9$ (all $p < 0.001$), with no monotonic trend in k.

We therefore recommend a simpler mitigation: source deployment transcripts from the same model family as the judge, or from a family with a similar $P(\text{eval})$ distribution on a held-out set. The correction presented in Section 3.3.3 can be used to recover most of the unconfounded signal from a mixed pool of deployment.